



**UNIVERSIDADE ESTADUAL DE MARINGÁ – UEM**  
**CENTRO DE TECNOLOGIA – CTC**  
**DEPARTAMENTO DE INFORMÁTICA – DIN**

EDUARDO ANGELO ROZADA MINHOLI – RA 134932

**RELATÓRIO TÉCNICO TRABALHO I DE PDI**  
**“INVERSÃO DE VALORES DE MATIZES”**

Maringá-PR  
2026

EDUARDO ANGELO ROZADA MINHOLI – RA 134932

**RELATÓRIO TÉCNICO TRABALHO I DE PDI**  
**“INVERSÃO DE VALORES DE MATIZES”**

*Relatório técnico do Trabalho I apresentado como requisito total à aprovação na disciplina de “Processamento Digital de Imagens” do Curso de Bacharelado em Ciência da Computação da Universidade Estadual de Maringá.*

Prof. Dr. Franklin Cesar Flores .

# Sumário

|                                                          | Páginas   |
|----------------------------------------------------------|-----------|
| <b>1 Introdução e Fundamentação Teórica</b>              | <b>4</b>  |
| 1.1 O Modelo RGB . . . . .                               | 4         |
| 1.2 O Espaço de Cores HSV . . . . .                      | 4         |
| 1.3 Descrição da Abordagem Algorítmica . . . . .         | 5         |
| <b>2 Implementação e Código-Fonte</b>                    | <b>5</b>  |
| 2.1 Módulo de Funções Auxiliares . . . . .               | 5         |
| 2.2 Módulo Principal de Processamento . . . . .          | 7         |
| 2.3 Arquivo de Execução . . . . .                        | 8         |
| <b>3 Resultados da Atividade e Casos de Teste</b>        | <b>9</b>  |
| 3.1 Caso de Teste 1: Cyberpunk Edgerunners . . . . .     | 10        |
| 3.2 Caso de Teste 2: Blade Runner 2049 . . . . .         | 11        |
| 3.3 Caso de Teste 3: Stray . . . . .                     | 12        |
| 3.4 Caso de Teste 4: Night City . . . . .                | 13        |
| <b>4 Discussão dos Resultados e Considerações Finais</b> | <b>14</b> |

# 1 Introdução e Fundamentação Teórica

Este projeto tem como objetivo explorar a manipulação de cores em imagens digitais através do processamento isolado de matizes de cor. Para compreender a abordagem utilizada no algoritmo, é necessário primeiramente diferenciar os modelos de representação de cores envolvidos no processo.

## 1.1 O Modelo RGB

O sistema RGB (Red, Green, Blue) é o padrão primário para captura e exibição de imagens em dispositivos digitais. Trata-se de um modelo aditivo, no qual diferentes intensidades de luz vermelha, verde e azul são combinadas para formar o espectro de cores visíveis.

Embora seja ideal para o hardware de monitores e câmeras, o RGB apresenta limitações para o processamento de imagens direcionado à cor. Nesse sistema, a informação da "cor" em si (matiz) está matematicamente atrelada à iluminação (brilho) e à intensidade (saturação). Dessa forma, tentar alterar uma cor específica manipulando diretamente os canais RGB, sem distorcer as sombras, os reflexos ou a naturalidade da imagem original, exige operações matemáticas complexas e sujeitas a artefatos visuais.

## 1.2 O Espaço de Cores HSV

Para contornar as limitações do RGB, utilizamos o espaço de cores HSV (Hue, Saturation, Value). Esse modelo representa as cores em um formato cilíndrico e foi desenvolvido para se aproximar da maneira como a visão humana percebe e descreve a luz. Ele decompõe a imagem em três canais distintos:

- **Matiz (Hue - H):** Representa o tipo de cor pura (como vermelho, amarelo, verde, ciano, azul ou magenta). É medida de forma angular, variando de 0 a 360 ao longo de um círculo cromático.
- **Saturação (Saturation - S):** Representa a pureza da cor, variando desde um tom totalmente acinzentado e neutro até a cor em sua intensidade máxima e viva.
- **Valor (Value - V):** Representa o brilho da cor, medindo a quantidade de luz que vai do preto absoluto até a iluminação total.

A principal vantagem do sistema HSV é o isolamento estrutural da cor na banda de Matiz (H). Isso permite que algoritmos alterem tonalidades de forma independente, preservando as bandas S e V, o que garante a manutenção da textura e da iluminação originais da cena.

### 1.3 Descrição da Abordagem Algorítmica

Baseando-se nessa fundamentação, o algoritmo desenvolvido atua recebendo uma imagem colorida e dois parâmetros de controle numéricos:  $H$  (a matiz central alvo, variando de 0 a 360) e  $d$  (a tolerância de variação, variando de 0 a 180).

O processo inicia-se com a conversão da imagem original do sistema RGB para o espaço HSV. Em seguida, a banda de Matiz é isolada e o algoritmo realiza uma varredura para identificar todos os pixels cuja matiz  $h$  se encontra dentro do intervalo de tolerância especificado, isto é,  $h \in [H - d, H + d]$ .

Quando a condição é satisfeita, a cor do pixel sofre uma rotação de 180 graus no círculo cromático, aplicando-se a operação  $h - 180$ . Como o canal H possui natureza circular contínua, o código implementa estruturas de normalização e módulos matemáticos para tratar adequadamente os cálculos em que os ângulos resultantes, ou os próprios limites do intervalo, cruzam as fronteiras do marco zero (0 ou 360 graus). Concluída a substituição de valores na matriz, os canais originais de saturação e brilho são reintegrados e a imagem resultante é convertida novamente para RGB, finalizando o processamento.

## 2 Implementação e Código-Fonte

A implementação foi desenvolvida em Python 3.x, utilizando a biblioteca OpenCV para carregar, converter e salvar as matrizes das imagens. O código foi dividido em três arquivos estruturados para manter o projeto limpo e modularizado.

### 2.1 Módulo de Funções Auxiliares

O arquivo `auxiliares.py` gerencia as operações básicas do programa. Para evitar erros de arredondamento e perdas de precisão nos cálculos angulares de  $[H - d, H + d]$ , as imagens são normalizadas temporariamente para o formato `float32` com o canal H mapeado diretamente em graus de 0 a 360.

```
1  # Arquivo para funcoes auxiliares
2
3  import cv2
4  import numpy as np
5
6  def normaliza_angulo(h):
7      """
8      Garante que um angulo fique no intervalo [0, 360).
9      """
10     return np.mod(h, 360)
11
12 def bgr_para_hsv(imagem_bgr):
13     """
```

```

14     Converte uma imagem BGR para HSV.
15
16     Trabalhamos com a imagem normalizada em float32 em vez do padrao do
17     opencv
18     porque, dessa maneira, o canal H e entregue diretamente em graus
19     (0-360),
20     preservando a precisao necessaria para os calculos do intervalo [H-d
21     , H+d].
22     """
23
24     img_float = imagem_bgr.astype(np.float32) / 255.0
25     hsv = cv2.cvtColor(img_float, cv2.COLOR_BGR2HSV)
26     return hsv
27
28 def hsv_para_bgr(imagem_hsv):
29     """
30     Converte uma imagem HSV (float32, H em [0,360), S e V em [0,1])
31     de volta para BGR.
32     """
33
34     bgr_float = cv2.cvtColor(imagem_hsv.astype(np.float32), cv2.
35         COLOR_HSV2BGR)
36     bgr_uint8 = np.clip(bgr_float * 255.0, 0, 255).astype(np.uint8)
37     return bgr_uint8
38
39 def carrega_imagem(caminho):
40     """Le uma imagem colorida do disco no formato BGR."""
41     imagem = cv2.imread(caminho, cv2.IMREAD_COLOR)
42     if imagem is None:
43         raise FileNotFoundError(f"Nao foi possivel abrir a imagem '{
44             caminho}'.")
45     return imagem
46
47 def salva_imagem(caminho, imagem_bgr):
48     """Salva uma imagem BGR no disco."""
49     cv2.imwrite(caminho, imagem_bgr)
50
51 def exibe_imagens(pares_titulo_imagem):
52     """
53     Exibe uma ou mais imagens em janelas separadas.
54
55     pares_titulo_imagem: lista de tuplas (titulo, imagem_bgr)
56     """
57     for titulo, imagem in pares_titulo_imagem:
58         cv2.imshow(titulo, imagem)
59     cv2.waitKey(0)
60     cv2.destroyAllWindows()

```

**Listing 1.** *auxiliares.py*

## 2.2 Módulo Principal de Processamento

O arquivo `inversao_matiz.py` isola os canais e faz a inversão da matiz na imagem, como também corrige o intervalo de H caso ele cruze a fronteira entre 360° e 0°.

```
1  # Arquivo para a inversao da Matiz
2
3  import numpy as np
4  from auxiliaries import normaliza_angulo, bgr_para_hsv, hsv_para_bgr
5
6  def valida_parametros(H, d):
7      """Valida os parametros H e d."""
8      if not (0 <= H <= 360):
9          raise ValueError("H deve estar entre 0 e 360.")
10     if not (0 <= d <= 180):
11         raise ValueError("d deve estar entre 0 e 180.")
12
13 def calcula_mascara_intervalo(H_band, H, d):
14     """
15     Calcula a mascara dos pixels cujo matiz h esta em [H-d, H+d],
16     tratando o caso em que o intervalo cruza a fronteira 0/360.
17     """
18     if d >= 180:
19         return np.ones_like(H_band, dtype=bool)
20
21
22     h_min = normaliza_angulo(H - d)
23     h_max = normaliza_angulo(H + d)
24
25     if h_min <= h_max:
26         mascara = (H_band >= h_min) & (H_band <= h_max)
27     else:
28         mascara = (H_band >= h_min) | (H_band <= h_max)
29
30     return mascara
31
32 def inverte_banda_h(H_band, H, d):
33     """
34     Recebe a banda H em graus, [0,360)) e devolve uma nova banda H
35     com os matizes do intervalo [H-d, H+d] substituidos por h - 180.
36     """
37     mascara = calcula_mascara_intervalo(H_band, H, d)
38
39     H_novo = H_band.copy()
40     H_novo[mascara] = normaliza_angulo(H_band[mascara] - 180)
41
42     return H_novo
```

```

43
44 def inverte_matizes(imagem_bgr, H, d):
45     """
46     Recebe uma imagem BGR e os parametros
47     H e d, e devolve uma nova imagem BGR com os matizes invertidos no
48     intervalo [H-d, H+d]. As bandas S e V sao preservadas.
49     """
50     valida_parametros(H, d)
51
52     # Converte para HSV
53     hsv = bgr_para_hsv(imagem_bgr)
54     H_band = hsv[:, :, 0]
55     S_band = hsv[:, :, 1]
56     V_band = hsv[:, :, 2]
57
58     # Processa somente a banda H
59     H_novo = inverte_banda_h(H_band, H, d)
60
61     # Reconstroi o HSV (S e V preservados) e converte de volta para BGR
62     hsv_novo = np.stack([H_novo, S_band, V_band], axis=-1)
63     return hsv_para_bgr(hsv_novo)

```

Listing 2. *inversao\_matiz.py*

## 2.3 Arquivo de Execução

O arquivo `main.py` atua como a interface de linha de comando, coletando os argumentos do terminal, invocando o fluxo principal e exibindo os resultados em tempo real.

```

1 import sys
2
3 from auxiliaries import carrega_imagem, salva_imagem, exibe_imagens
4 from inversao_matiz import inverte_matizes
5
6 def main():
7     if len(sys.argv) != 5:
8         print("Uso: python src/main.py <entrada> <saida> <H> <d>")
9         sys.exit(1)
10
11     caminho_entrada = sys.argv[1]
12     caminho_saida = sys.argv[2]
13     H = float(sys.argv[3])
14     d = float(sys.argv[4])
15
16     imagem = carrega_imagem(caminho_entrada)
17     resultado = inverte_matizes(imagem, H, d)
18

```

```

19     salva_imagem(caminho_saida, resultado)
20     print(f"Imagem salva em: {caminho_saida}")
21
22     exibe_imagens([("Original", imagem), ("Matizes invertidas",
23                                     resultado)])
24
25 if __name__ == "__main__":
26     main()

```

**Listing 3.** *main.py*

### 3 Resultados da Atividade e Casos de Teste

Para validar a eficiência e o comportamento do programa frente a diferentes distribuições cromáticas, foram realizados experimentos práticos utilizando quatro imagens distintas. Cada imagem foi submetida a quatro configurações diferentes das variáveis  $H$  e  $d$ .

Abaixo, os resultados estão organizados de forma visual, exibindo a imagem original centralizada no topo e suas respectivas variações dispostas lado a lado logo abaixo.

### 3.1 Caso de Teste 1: Cyberpunk Edgerunners



Figura 1. Imagem original: Cyberpunk Edgerunners.



(a) Teste 1:  $H = 60, d = 25$



(b) Teste 2:  $H = 0, d = 15$



(c) Teste 3:  $H = 300, d = 20$



(d) Teste 4:  $H = 180, d = 160$

Figura 2. Resultados obtidos na Imagem 1 através da inversão adaptativa de matizes.

### 3.2 Caso de Teste 2: Blade Runner 2049



**Figura 3.** Imagem original: *Blade Runner 2049*.



(a) Teste 1:  $H = 315, d = 30$



(b) Teste 2:  $H = 225, d = 35$



(c) Teste 3:  $H = 275, d = 15$



(d) Teste 4:  $H = 240, d = 60$

**Figura 4.** Resultados obtidos na Imagem 2 evidenciando o comportamento com gradientes suaves.

### 3.3 Caso de Teste 3: Stray



**Figura 5.** Imagem original: Stray.



(a) Teste 1:  $H = 195, d = 20$



(b) Teste 2:  $H = 55, d = 25$



(c) Teste 3:  $H = 315, d = 25$



(d) Teste 4:  $H = 30, d = 45$

**Figura 6.** Resultados obtidos na Imagem 3 analisando o isolamento de canais de luz neon.

### 3.4 Caso de Teste 4: Night City



**Figura 7.** Imagem original: Night City.



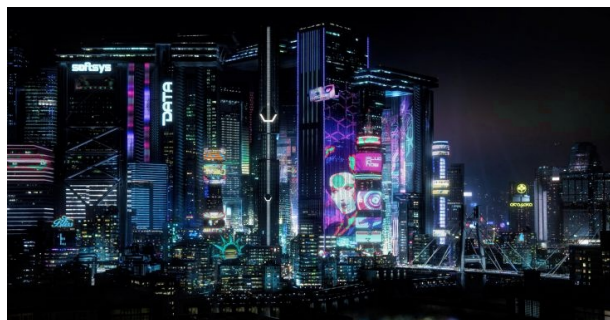
(a) Teste 1:  $H = 210, d = 30$



(b) Teste 2:  $H = 300, d = 25$



(c) Teste 3:  $H = 60, d = 15$



(d) Teste 4:  $H = 0, d = 20$

**Figura 8.** Resultados obtidos na Imagem 4 em um cenário urbano noturno complexo.

## 4 Discussão dos Resultados e Considerações Finais

A análise prática dos experimentos trouxe observações sobre como o espaço de cores HSV se comporta no mundo real. Nos testes envolvendo gradientes suaves (como em *Blade Runner 2049*), pudemos notar o aparecimento de contornos serrilhados nas regiões de transição. Esse fenômeno acontece porque o algoritmo toma uma decisão estritamente binária com base no parâmetro  $d$ : ou o pixel está dentro do intervalo e muda completamente, ou fica de fora e mantém a cor original.

Outro ponto que chamou a atenção foi a alteração de tonalidade em áreas muito escuras ou reflexivas, como o pelo do gato na imagem do jogo *Stray*. Mesmo que uma superfície pareça preta ou cinza a olho nu, ela ainda carrega uma assinatura de matiz residual provocada pela luz ambiente do cenário.

Por fim, gostaria de dizer que foi um trabalho muito legal de se desenvolver, ainda mais quando existe a possibilidade de utilizar conteúdos dos quais você é muito fã como casos de teste e ver tudo realmente funcionando no final. Por favor, continue passando trabalhos assim para as próximas turmas!